Indexes - Optimize Queries

We just saw how to get some data `WHERE` a **condition** is true. For example, where the hero **name is "Deadpond"**.

If we just create the tables and the data as we have been doing, when we `SELECT` some data using `WHERE`, the database would have to **scan** through **each one of the records** to find the ones that **match**. This is not a problem with 3 heroes as in these examples.

But imagine that your database has **thousands** or **millions** of **records**, if every time you want to find the heroes with the name "Deadpond" it has to scan through **all** of the records to find all the possible matches, then that becomes problematic, as it would be too slow.

I'll show you how to handle it with a database **index**.

The change in the code is **extremely small**, but it's useful to understand what's happening behind the scenes, so I'll show you **how it all works** and what it means.

If you already executed the previous examples and have a database with data, **remove the database file** before running each example, that way you won't have duplicate data and you will be able to get the same results.

No Time to Explain

Are you already a **SQL expert** and don't have time for all my explanations?

Fine, in that case, you can **sneak peek** the final code to create indexes here.

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age <= 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

..but if you are not an expert, **continue reading**, this will probably be useful. 🤓

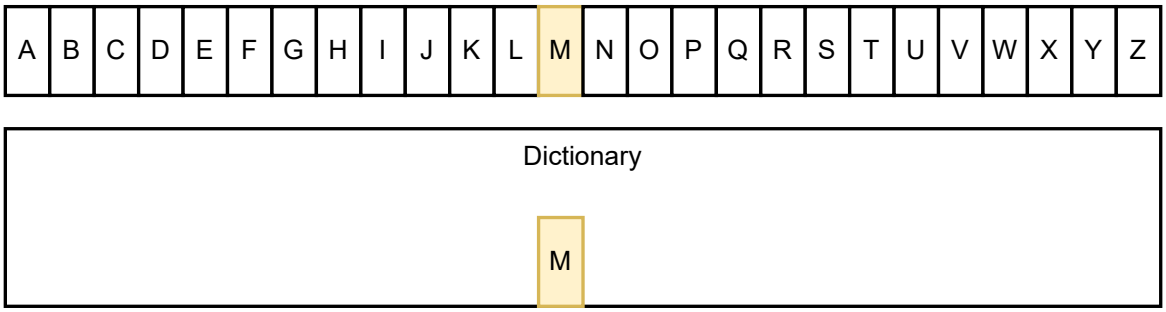
What is an Index

In general, an **index** is just something we can have to help us **find things faster**. It normally works by having things in **order**. Let's think about some real-life examples before even thinking about databases and code.

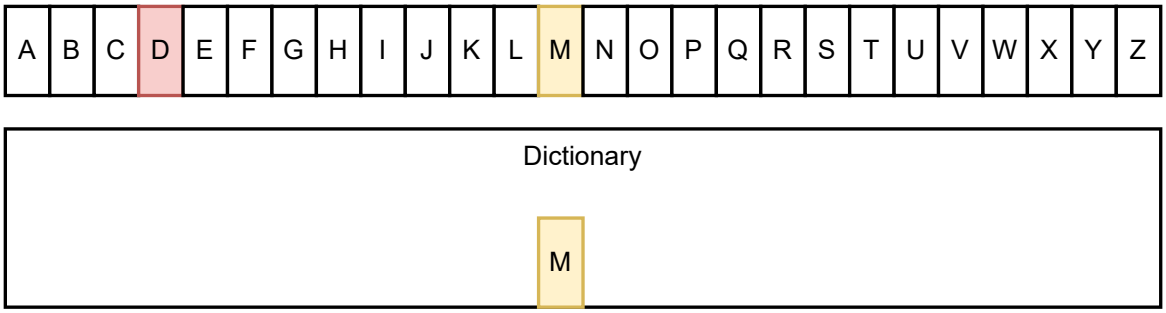
An Index and a Dictionary

Imagine a **dictionary**, a book with definitions of words. 📔 ...not a Python `dict`. 😅

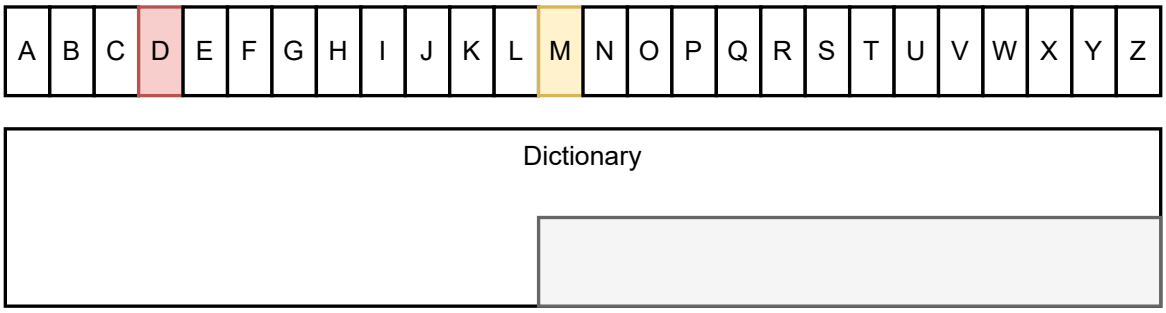
Let's say that you want to **find a word**, for example the word "**database**". You take the dictionary, and open it somewhere, for example in the middle. Maybe you see some definitions of words that start with `m`, like `manual`, so you conclude that you are in the letter `m` in the dictionary.



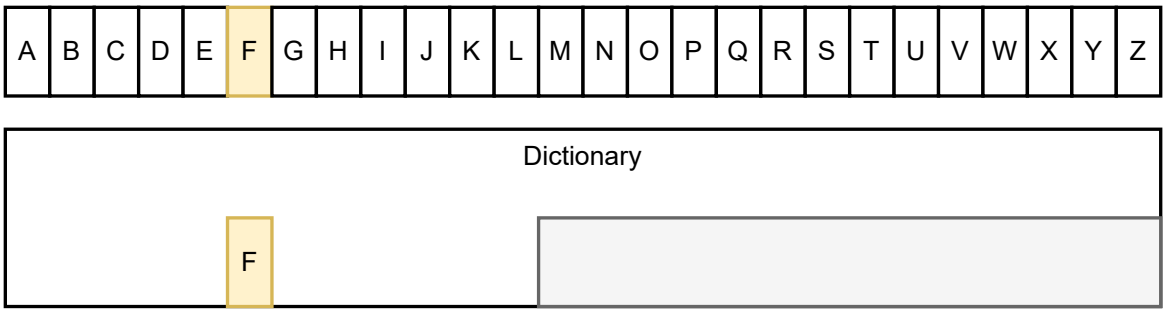
You know that in the alphabet, the letter `d` for `database` comes **before** the letter `m` for `manual`.



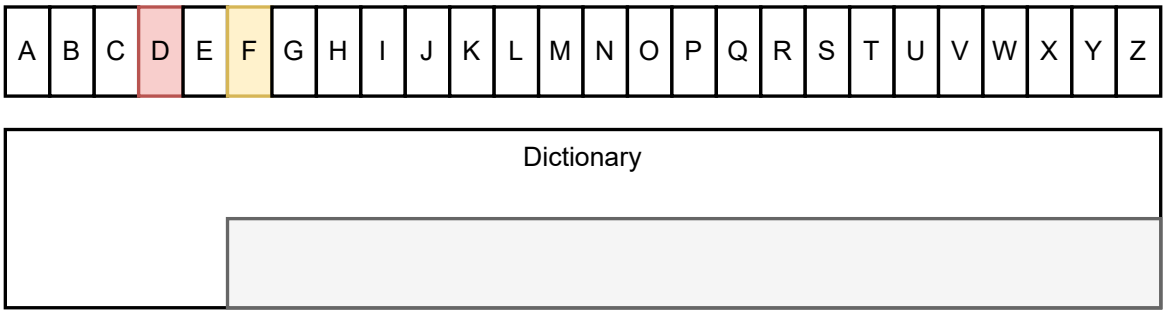
So, you know you have to search in the dictionary **before** the point you currently are. You still don't know where the word `database` is, because you don't know exactly where the letter `d` is in the dictionary, but you know that **it is not after** that point, you can now **discard the right half** of the dictionary in your search.



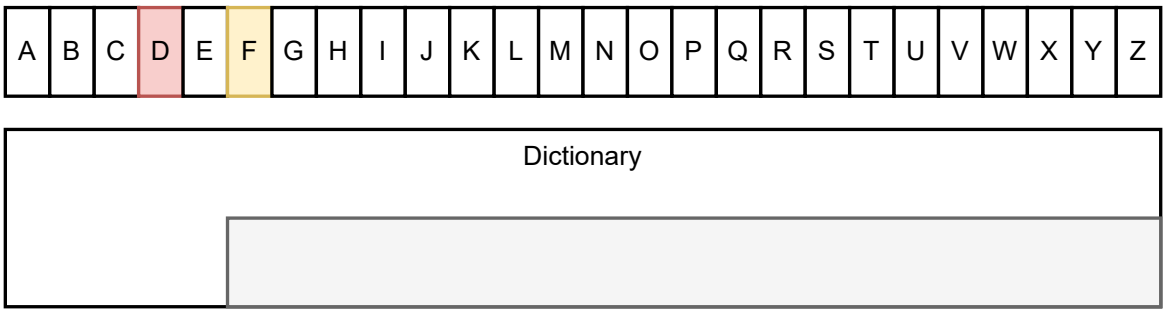
Next, you **open the dictionary again**, but only taking into account the **half of the dictionary** that can contain the word you want, the **left part of the dictionary**. You open it in the middle of that left part and now you arrive maybe at the letter `f`.



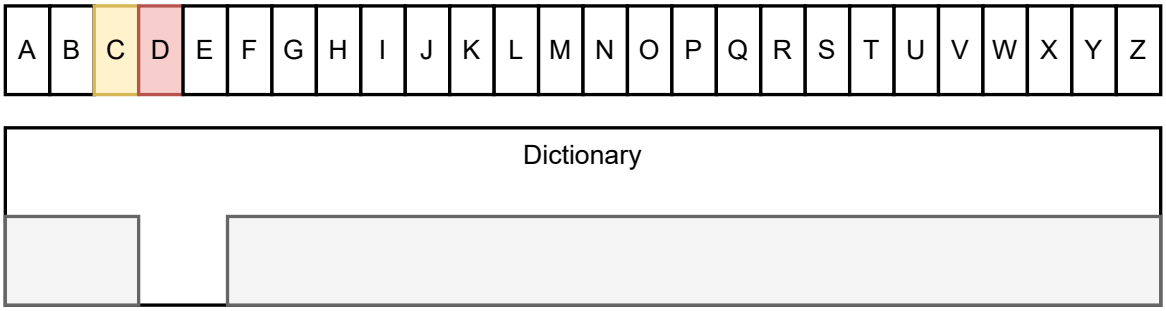
You know that `d` from `database` comes before `f`. So it has to be **before** that. But now you know that `database` **is not after** that point, and you can discard the dictionary from that point onward.



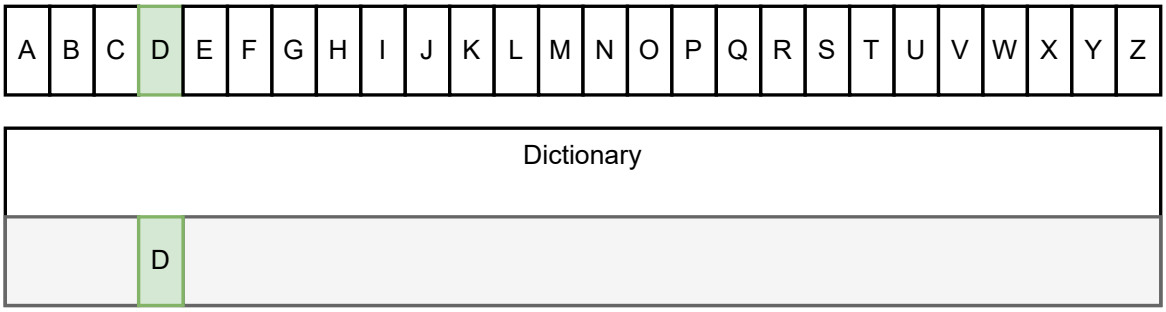
Now you have a **small section of dictionary** to search (only a **quarter** of dictionary can have your word). You take that **quarter** of the pages at the start of the dictionary that can contain your word, and open it in the middle of that section. Maybe you arrive at the letter `c`.



You know the word `database` has to be **after** that and **not before** that point, so you can discard the left part of that block of pages.



You repeat this process **a few more times**, and you finally arrive at the letter `d`, you continue with the same process in that section for the letter `d` and you finally **find the word** `database`. 🎉



You had to open the dictionary a few times, maybe **5 or 10**. That's actually **very little work** compared to what it could have been.

Technical Details

Do you like **fancy words**? Cool! Programmers tend to like fancy words. 😅

That algorithm I showed you above is called **Binary Search**.

It's called that because you **search** something by splitting the dictionary (or any ordered list of things) in **two** ("binary" means "two") parts. And you do that process multiple times until you find what you want.

An Index and a Novel

Let's now imagine you are reading a **novel book**. And someone told you that at some point, they mention a **database**, and you want to find that chapter.

How do you find the word "*database*" there? You might have to read **the entire book** to find where the word "*database*" is located in the book. So, instead of opening the book 5 or 10 times, you would have to open each of the **500 pages** and read them one by one until you find the word. You might enjoy the book, though. 😅

But if we are only interested in **quickly finding information** (as when working with SQL databases), then reading each of the 500 pages is **too inefficient** when there could be an option to open the book in 5 or 10 places and find what you're looking for.

A Technical Book with an Index

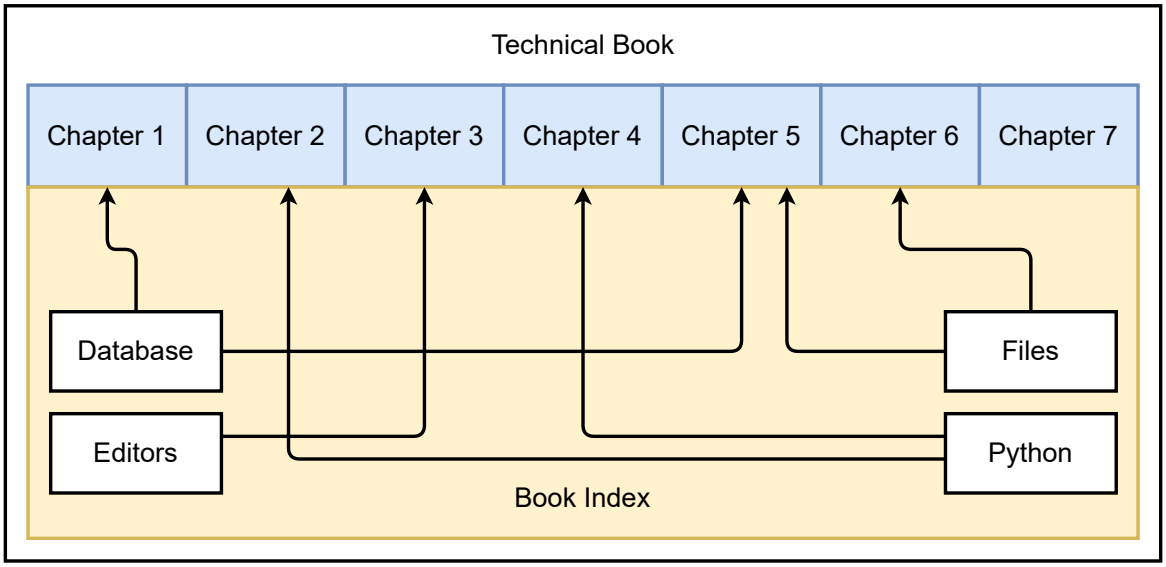
Now let's imagine you are reading a technical book. For example, with several topics about programming. And there's a couple of sections where it talks about a **database**.

This book might have a **book index**: a section in the book that has some **names of topics covered** and the **page numbers** in the book where you can read about them. And the topic names are **sorted** in alphabetic order, pretty much like a dictionary (a book with words, as in the previous example).

In this case, you can open that book in the end (or in the beginning) to find the **book index** section, it would have only a few pages. And then, you can do the same process as with the **dictionary** example above.

Open the index, and after **5 or 10 steps**, quickly find the topic "**database**" with the page numbers where that is covered, for example "page 253 in Chapter 5". Now you used the dictionary technique to find the **topic**, and that topic gave you a **page number**.

Now you know that you need to find "**page 253**". But by looking at the closed book you still don't know where that page is, so you have to **find that page**. To find it, you can do the same process again, but this time, instead of searching for a **topic** in the **index**, you are searching for a **page number** in the **entire book**. And after **5 or 10 more steps**, you find the page 253 in Chapter 5.



After this, even though this book is not a dictionary and has some particular content, you were able to **find the section** in the book that talks about a "**database**" in a **few steps** (say 10 or 20, instead of reading all the 500 pages).

The main point is that the index is **sorted**, so we can use the same process we used for the **dictionary** to find the topic. And then that gives us a page number, and the **page numbers are also sorted**! 😄

When we have a list of sorted things we can apply the same technique, and that's the whole trick here, we use the same technique first for the **topics** in the index and then for the **page numbers** to find the actual chapter.

Such efficiency! 😎

What are Database Indexes

Database indexes are very similar to **book indexes**.

Database indexes store some info, some keys, in a way that makes it **easy and fast to find** (for example sorted), and then for each key they **point to some data somewhere else** in the database.

Let's see a more clear example. Let's say you have this table in a database:

id	name	secret_name	age
1	Deadpond	Dive Wilson	null
2	Spider-Boy	Pedro Parqueador	null
3	Rusty-Man	Tommy Sharp	48

And let's imagine you have **many more rows**, many more heroes. Probably **thousands**.

If you tell the SQL database to get you a hero by a specific name, for example `Spider-Boy` (by using the `name` in the `WHERE` part of the SQL query), the database will have to **scan** all the heroes, checking **one by one** to find all the ones with a name of `Spider-Boy`.

In this case, there's only one, but there's nothing limiting the database from having **more records with the same name**. And because of that, the database would **continue searching** and checking each one of the records, which would be very slow.

But now let's say that the database has an index for the column `name`. The index could look something like this, we could imagine that the index is like an additional special table that the database manages automatically:

name	id
Deadpond	1
Rusty-Man	3
Spider-Boy	2

It would have each `name` field from the `hero` table **in order**. It would not be sorted by `id`, but by `name` (in alphabetical order, as the `name` is a string). So, first it would have `Deadpond`, then `Rusty-Man`, and last `Spider-Boy`. It would also include the `id` of each hero. Remember that this could have **thousands** of heroes.

Then the database would be able to use more or less the same ideas in the examples above with the **dictionary** and the **book index**.

It could start somewhere (for example, in the middle of the index). It could arrive at some hero there in the middle, like `Rusty-Man`. And because the **index** has the `name` fields in order, the database would know that it can **discard all the previous index rows** and **only search** in the following index rows.

name	id
Deadpond	1
Rusty-Man	3
Spider-Boy	2

And that way, as with the example with the dictionary above, **instead of reading thousands of heroes**, the database would be able to do a few steps, say **5 or 10 steps**, and arrive at the row of the index that has `Spider-Boy`, even if the table (and index) has thousands of rows:

name	id
Deadpond	1
Rusty-Man	3
✨ Spider-Boy ✨	2

Then by looking at **this index row**, it would know that the `id` for `Spider-Boy` in the `hero` table is `2`.

So then it could **search that `id`** in the `hero` table using more or less the **same technique**.

That way, in the end, instead of reading thousands of records, the database only had to do **a few steps** to find the hero we wanted.

Updating the Index

As you can imagine, for all this to work, the index would need to be **up to date** with the data in the database.

If you had to update it **manually** in code, it would be very cumbersome and **error-prone**, as it would be easy to end up in a state where the index is not up to date and points to incorrect data. 😱

Here's the good news: when you create an index in a **SQL Database**, the database takes care of **updating** it **automatically** whenever it's necessary. 😎🎉

If you **add new records** to the `hero` table, the database will **automatically** update the index. It will do the **same process** of **finding** the right place to put the new index data (those **5 or 10 steps** described above), and then it will save the new index information there. The same would happen when you **update** or **delete** data.

Defining and creating an index is very **easy** with SQL databases. And then **using it** is even easier... it's transparent. The database will figure out which index to use automatically, the SQL queries don't even change.

So, in SQL databases **indexes are great**! And are super **easy to use**. Why not just have indexes for everything? .....Because indexes also have a "**cost**" in computation and storage (disk space).

Index Cost

There's a **cost** associated with **indexes**. 💰

When you don't have an index and add a **new row** to the table `hero`, the database has to perform **1 operation** to add the new hero row at the end of the table.

But if you have an **index** for the **hero names**, now the database has to perform the same **1 operation** to add that row **plus** some extra **5 or 10 operations** in the index, to find the right spot for the name, to then add that **index record** there.

And if you have an index for the `name`, one for the `age`, and one for the `secret_name`, now the database has to perform the same **1 operation** to add that row **plus** some extra **5 or 10 operations** in the index **times 3**, for each of the indexes. This means that now adding one row takes something like **31 operations**.

This also means that you are **exchanging** the time it takes to **read** data for the time it takes to **write** data plus some extra **space** in the database.

If you have queries that get data out of the database comparing each one of those fields (for example using `WHERE`), then it makes total sense to have indexes for each one of them. Because **31 operations** while creating or updating data (plus the space of the index) is much, much better than the possible **500 or 1000 operations** to read all the rows to be able to compare them using each field.

But if you **never** have queries that find records by the `secret_name` (you never use `secret_name` in the `WHERE` part) it probably doesn't make sense to have an index for the `secret_name` field/column, as that will increase the computational and space **cost** of writing and updating the database.

Create an Index with SQL

Phew, that was a lot of theory and explanations. 😅

The most important thing about indexes is **understanding** them, how, and when to use them.

Let's now see the **SQL** syntax to create an **index**. It is very simple:

```
CREATE INDEX ix_hero_name
ON hero (name)
```

This means, more or less:

> Hey SQL database 👋, please `CREATE` an `INDEX` for me.

> I want the name of the index to be `ix_hero_name`.
>
> This index should be `ON` the table `hero`, it refers to that table.
>
> The column I want you to use for it is `name`.

Declare Indexes with SQLModel

And now let's see how to define indexes in **SQLModel**.

The change in code is underwhelming, it's very simple. 😆

Here's the `Hero` model we had before:

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str
    secret_name: str
    age: int | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Deadpond")
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Let's now update it to tell **SQLModel** to create an index for the `name` field when creating the table:

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Deadpond")
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

We use the same `Field()` again as we did before, and set `index=True`. That's it! 🚀

Notice that we didn't set an argument of `default=None` or anything similar. This means that **SQLModel** (thanks to Pydantic) will keep it as a **required** field.

Info

SQLModel (actually SQLAlchemy) will **automatically generate the index name** for you.

In this case the generated name would be `ix_hero_name`.

Query Data

Now, to query the data using the field `name` and the new index we don't have to do anything special or different in the code, it's just **the same code**.

The SQL database will figure it out **automatically**. ✨

This is great because it means that indexes are very **simple to use**. But it might also feel counterintuitive at first, as you are **not doing anything** explicitly in the code to make it obvious that the index is useful, it all happens in the database behind the scenes.

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Deadpond")
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Deadpond")
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

This is exactly the same code as we had before, but now the database will **use the index** underneath.

Run the Program

If you run the program now, you will see an output like this:

```bash
```

More Indexes

We are going to query the `hero` table doing comparisons on the `age` field too, so we should **define an index** for that one as well:

Python 3.10+

```
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age <= 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

In this case, we want the default value of `age` to continue being `None`, so we set `default=None` when using `Field()`.

Now when we use **SQLModel** to create the database and tables, it will also create the **indexes** for these two columns in the `hero` table.

So, when we query the database for the `hero` table and use those **two columns** to define what data we get, the database will be able to **use those indexes** to improve the **reading performance**. 🚀

Primary Key and Indexes

You probably noticed that we didn't set `index=True` for the `id` field.

Because the `id` is already the **primary key**, the database will automatically create an internal **index** for it.

The database always creates an internal index for **primary keys** automatically, as those are the primary way to organize, store, and retrieve data. 🤓

But if you want to be **frequently querying** the SQL database for any **other field** (e.g. using any other field in the `WHERE` section), you will probably want to have at least an **index** for that.

Recap

Indexes are very important to improve **reading performance** and speed when querying the database. 🏎️

Creating and using them is very **simple** and easy. The most important part is to understand **how** they work, **when** to create them, and for **which columns**.